

FinsightAI Backend Architecture

Detailed File-by-File Technical Directory

Overview

This guide outlines the production-ready project structure of FinsightAI. Each file and folder serves a unique purpose in maintaining the Clean Architecture pattern, ensuring scalability, ease of testing, and modular separation of business rules, AI agents, and RAG pipelines.

1. Infrastructure & Configurations

[requirements.txt](#)

Description: A list of all the third-party Python packages our app needs (FastAPI, SQLAlchemy, Celery, LangChain, etc.).

[.env.example](#)

Description: A template listing all the environment configuration options (like database credentials and API keys) needed locally.

[.gitignore](#)

Description: Tells Git which files to ignore (like private credentials, temporary variables, and virtual environment files).

[Dockerfile](#)

Description: A set of blueprint instructions to build our FastAPI application inside a standard Docker container.

[docker-compose.yml](#)

Description: Coordinates multiple Docker containers running simultaneously, including our web app, PostgreSQL DB, Redis cache, and Celery task worker.

[alembic.ini](#)

Description: Configuration file for Alembic, telling it where to find migration versions and how to connect to the database.

[README.md](#)

Description: The project manual detailing structural breakdown, running instructions, and how to execute test suites.

2. Database & Data Models

[alembic/env.py](#)

Description: Bootstraps migration configurations, dynamically fetching database credentials from settings and executing migrations asynchronously.

[alembic/script.py.mako](#)

Description: Template used by Alembic to automatically generate new migration version files.

[app/db/session.py](#)

Description: Sets up the async database engine and provides db sessions to our routes using FastAPI dependency injection.

FinsightAI Backend Architecture

Detailed File-by-File Technical Directory

[app/db/base.py](#)

Description: Central registry importing all database models so Alembic can track model changes and autogenerate schemas.

[app/models/base.py](#)

Description: Declares the base SQLAlchemy class and an abstract base model giving every table a UUID primary key and timestamp tracking.

3. Application Core Modules

[app/main.py](#)

Description: The app's entrypoint initialization file. Configures CORS, global exception filters, logging setups, and mounts API routers.

[app/core/config.py](#)

Description: Retrieves system config from environment variables using Pydantic Settings v2, and provides type-validated config constants.

[app/core/logging.py](#)

Description: Configures centralized logging via Loguru, intercepting system logs from libraries like Uvicorn and SQLAlchemy to keep stdout consistent.

[app/core/exceptions.py](#)

Description: Defines customized domain errors and overrides default error returns with clean, structured HTTP JSON responses.

[app/core/celery_app.py](#)

Description: Initializes the Celery background worker, setting up Redis as the broker to process long-running jobs in the background.

FinsightAI Backend Architecture

Detailed File-by-File Technical Directory

4. Business Rules, Services, & Data Access

[app/repositories/base.py](#)

Description: An abstract database helper implementing generic CRUD methods (Get, Create, Update, Delete) to decouple SQL operations from business rules.

[app/services/base.py](#)

Description: Base business logic service layer coordinating database repositories and processing domain workflows.

[app/schemas/__init__.py](#)

Description: Houses Pydantic data serialization schemas to structure and validate HTTP request payloads and API JSON responses.

5. API Endpoints & Routes

[app/api/v1/api.py](#)

Description: Unifies separate domain sub-routers into a single cohesive v1 API router bundle.

[app/api/v1/routers/health.py](#)

Description: Exposes a '/health' check route that runs real-time connectivity validation on PostgreSQL and Redis states.

[app/api/v1/routers/\[auth, chat, company, document, event, news\].py](#)

Description: Separate modular routers serving as entry points for each system feature (Authentication, AI Agents, Financials, RAG documents, etc.).

6. AI, RAG & Modules

[app/ai/](#)

Description: Modular folder with subdirectories for LangGraph orchestrators, specialized agents, prompt templates, memory stores, and custom agent tools.

[app/rag/](#)

Description: Contains data loading, parser, text chunking, embedding generation, retrieval rules, and vector store configurations.

7. Testing & Quality Assurance

[tests/conftest.py](#)

Description: Defines test fixtures and swaps production databases with mock database instances during testing.

[tests/test_main.py](#)

FinsightAI Backend Architecture

Detailed File-by-File Technical Directory

Description: Runs test assertions on our endpoints to ensure routes register properly and health checks respond correctly.